

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОНИКИ И МАТЕМАТИКИ
им. А.Н. Тихонова НИУ ВШЭ

ДЕПАРТАМЕНТ КОМПЬЮТЕРНОЙ ИНЖЕНЕРИИ

КУРСОВАЯ РАБОТА
по дисциплине «Алгоритмизация и программирование»

**Тема: «Разработка приложения для пакетной конвертации видеороликов в
эффективный формат хранения»**

Выполнил:
студент группы БИВ253
Солодилов Михаил Анатольевич

Руководитель:
Волков Владимир Олегович

Москва, 2026

Задание на курсовую работу

Тема: реализовать программу для пакетной конвертации видеороликов в более эффективный формат для сокращения занимаемого места на диске.

Цель работы: разработать настольное приложение Floppy, позволяющее пользователю выбрать один или несколько видеофайлов и перекодировать их в формат HEVC с настройкой качества, разрешения и частоты кадров. Итоговая программа должна уменьшать размер видеофайлов при сохранении приемлемого качества изображения.

Задачи работы:

- 1) изучить предметную область хранения и перекодирования видеороликов;
- 2) рассмотреть существующие инструменты и подходы к сжатию видео;
- 3) выбрать технологический стек для реализации настольного приложения;
- 4) реализовать пользовательский интерфейс для выбора файлов и настройки параметров перекодирования;
- 5) реализовать модуль перекодирования через FFmpeg;
- 6) добавить пакетную обработку файлов и отображение прогресса;
- 7) провести тестирование основных сценариев работы программы;
- 8) подготовить пользовательскую и разработческую документацию.

Функциональные требования:

- 1) программа должна позволять выбрать один или несколько видеофайлов через диалог выбора файлов;
- 2) программа должна поддерживать добавление файлов перетаскиванием в окно приложения;
- 3) программа должна принимать на вход локальные видеофайлы, доступные файловой системе пользователя;
- 4) программа должна позволять указать параметр качества перекодирования;

- 5) программа должна позволять ограничить высоту выходного видео;
- 6) программа должна позволять ограничить частоту кадров выходного видео;
- 7) программа должна создавать выходной файл с суффиксом `_compressed`;
- 8) программа должна отображать прогресс выполнения операции;
- 9) программа должна сообщать пользователю об ошибке, если перекодирование завершилось неуспешно.

Нефункциональные требования:

- 1) язык реализации: Python;
- 2) графический интерфейс: GTK 4 через PyGObject;
- 3) обработка видео: FFmpeg и библиотека python-ffmpeg;
- 4) платформа: настольная операционная система Linux с установленными FFmpeg и GTK 4;
- 5) формат хранения данных: локальные файлы пользователя, база данных не используется.

Ожидаемые результаты:

- 1) рабочее приложение, выполняющее пакетное перекодирование видео;
- 2) исходный код проекта;
- 3) инструкция по запуску и использованию;
- 4) примеры входных данных и результата перекодирования.

График выполнения

№	Этап работы	Дата
1	Обоснование темы, описание предметной области, постановка цели и задач	29.11.2025
2	Короткий обзор аналогов и источников литературы	13.12.2025
3	Подготовка окончательной версии технического задания	20.12.2025
4	Реализация базового модуля перекодирования через FFmpeg	08.05.2026
5	Реализация графического интерфейса и пакетной обработки файлов	09.05.2026
6	Тестирование приложения и подготовка документации	15.05.2026

Аннотация

В курсовой работе рассматривается разработка настольного приложения Floppy для пакетной конвертации видеороликов в формат HEVC. Объектом разработки является программное средство для уменьшения занимаемого видеофайлами места на диске. Цель работы заключается в создании приложения с графическим интерфейсом, которое позволяет выбрать несколько видеофайлов, настроить качество, разрешение и частоту кадров, запустить перекодирование и получить выходные файлы с суффиксом `_compressed`.

В ходе работы были изучены подходы к сжатию видео, выбран стек Python, GTK 4, PyGObject, FFmpeg и python-ffmpeg, реализованы модули пользовательского интерфейса, выбора параметров, определения доступного HEVC-кодека, запуска FFmpeg и отображения прогресса. Результатом стала программа, предназначенная для локальной обработки видеофайлов без использования сетевых сервисов и базы данных.

Работа содержит 19 страниц, 0 иллюстраций, 1 таблицу, 4 использованных источника.

ОГЛАВЛЕНИЕ

1	Введение	7
2	Обзор аналогов и источников	8
3	Описание разработанного приложения	9
4	Архитектура и реализация	10
4.1	Графический интерфейс	10
4.2	Пакетная обработка	10
4.3	Модуль перекодирования	10
4.4	Выбор кодировщика	11
5	Ход разработки	12
6	Тестирование	13
7	Заключение	14
8	Список использованных источников	15
A	Руководство пользователя	16
A.1	Назначение программы	16
A.2	Установка	16
A.3	Запуск	16
A.4	Основной сценарий использования	16
A.5	Типовые ошибки	17
B	Руководство разработчика	18
B.1	Структура проекта	18
B.2	Поток выполнения	18
B.3	Выбор кодировщика	18
B.4	Сборка и запуск при разработке	19
B.5	Возможные направления развития	19

1 Введение

Объем видеоданных, которые пользователи хранят на персональных компьютерах и внешних накопителях, постоянно увеличивается. Видеоролики с мобильных устройств, камер и экранных записей часто сохраняются в высоком разрешении и занимают значительное место на диске. При этом часть таких файлов используется только для архива, пересылки или просмотра, поэтому для них важнее рациональный размер, чем слишком высокое качество.

Одним из способов уменьшения размера видео является перекодирование в более эффективный видеокодек. В данной работе используется формат HEVC, также известный как H.265. Он позволяет добиться более высокой степени сжатия по сравнению с рядом старых форматов при сопоставимом визуальном качестве [1]. Для практической обработки видео выбран FFmpeg, так как он является стандартом в индустрии для кодирования, декодирования и преобразования мультимедийных файлов [2].

Актуальность темы связана с тем, что работа с FFmpeg требует специфических знаний об устройстве кодеков и о возможностях разных аппаратных ускорителей. Итого, прежде чем понять, как на конкретном компьютере пользоваться FFmpeg, нужно довольно глубоко погрузиться в документацию этой утилиты. Поэтому целесообразно разработать графическое приложение, которое скрывает низкоуровневые команды и предоставляет понятные элементы управления.

Целью курсовой работы является разработка приложения Floppy для пакетной конвертации видеороликов в эффективный формат хранения. Для достижения цели необходимо решить следующие задачи:

- определить требования к приложению;
- изучить инструменты перекодирования видео;
- реализовать выбор файлов и параметров обработки;
- реализовать запуск FFmpeg из Python-приложения;
- обеспечить отображение прогресса и сообщений о результате;
- провести тестирование основных пользовательских сценариев.

2 Обзор аналогов и источников

Для решения задачи уменьшения размера видео существуют как универсальные консольные инструменты, так и графические приложения.

FFmpeg является наиболее гибким инструментом. Он поддерживает большое количество контейнеров, кодеков, фильтров и аппаратных кодировщиков [2]. Его преимуществом является высокая универсальность, а недостатком для рядового пользователя является сложность командной строки.

HandBrake является популярным графическим приложением для перекодирования видео. Оно предоставляет готовые профили, настройки качества и очереди задач. Сильной стороной такого подхода является развитый интерфейс, но для учебного проекта избыточность HandBrake показывает, что можно реализовать более простое приложение, ориентированное только на пакетное сжатие выбранных файлов.

Для реализации интерфейса Floppy используется GTK 4 через PyGObject. GTK предоставляет виджеты для окон, кнопок, полей ввода, диалогов выбора файлов и индикаторов прогресса [3]. Для асинхронной работы с FFmpeg выбрана библиотека `python-ffmpeg`, которая позволяет запускать обработку из Python-кода и получать события прогресса [4].

В результате анализа был выбран подход, при котором графический интерфейс отвечает за взаимодействие с пользователем, а обработка видео выполняется через FFmpeg. Такой вариант позволяет не реализовывать кодирование самостоятельно, а использовать готовый и проверенный инструмент.

3 Описание разработанного приложения

Разработанное приложение Floppy представляет собой настольную программу с графическим интерфейсом. Оно предназначено для пакетного перекодирования видеороликов в HEVC. Пользователь выбирает один или несколько файлов, задает параметры обработки и запускает кодирование. Для каждого входного файла создается отдельный выходной файл с суффиксом `_compressed`.

Проект состоит из следующих основных файлов:

- `app.py` – модуль графического интерфейса и пользовательских сценариев;
- `engine.py` – модуль запуска перекодирования;
- `utils.py` – вспомогательный модуль для работы с FFmpeg, FFprobe и параметрами кодировщиков;
- `requirement.txt` – список Python-зависимостей проекта.

Пользовательский интерфейс реализован в классе `MainWindow`. Окно содержит название приложения, строку выбранных файлов, кнопку выбора файлов, подсказку о перетаскивании, поля настройки качества, разрешения и частоты кадров, индикатор прогресса, строку статуса и кнопку запуска перекодирования.

4 Архитектура и реализация

4.1 Графический интерфейс

Модуль `app.py` создает GTK-приложение с идентификатором `dev.floppy.Reencoder`. При активации приложения создается главное окно. В интерфейсе используется вертикальное расположение элементов: сверху находится заголовок, далее идут элементы выбора файлов и настройки параметров, затем индикатор прогресса и кнопка запуска.

Выбор файлов реализован через `Gtk.FileChooserNative`. Диалог поддерживает множественный выбор. Дополнительно в окно добавлен обработчик перетаскивания файлов через `Gtk.DropTarget`. Это позволяет пользователю выбрать видео как через стандартный диалог, так и прямым переносом файлов в окно.

Для настройки качества используется числовое поле со значениями от 1 до 51. Разрешение и частота кадров также задаются числовыми полями. Значение 0 для разрешения означает сохранение исходного размера кадра, а значение 0 для частоты кадров означает сохранение исходной частоты кадров.

4.2 Пакетная обработка

После нажатия кнопки `Reencode` приложение проверяет, выбраны ли файлы. Если список пуст, пользователь получает сообщение `Choose files first`. Если файлы выбраны, интерфейс блокирует кнопки выбора и запуска, чтобы пользователь не начал вторую обработку параллельно.

Перекодирование выполняется в отдельном потоке. Это важно для графического интерфейса: длительная операция `FFmpeg` не должна блокировать основной поток GTK. После обработки каждого файла статус обновляется, а прогресс устанавливается в 100 процентов. При возникновении ошибки приложение разблокирует кнопки и показывает сообщение с количеством уже обработанных файлов.

4.3 Модуль перекодирования

Модуль `engine.py` содержит асинхронную функцию `reencode`. Она принимает путь к входному файлу, параметр качества, ограничение разрешения, ограничение частоты кадров и необязательную функцию обратного вызова для прогресса.

Перед запуском FFmpeg модуль определяет исходное разрешение и частоту кадров. Если пользовательское ограничение выше или равно исходному значению, параметр не применяется. Это предотвращает бессмысленное увеличение разрешения или частоты кадров при сжатии.

Имя выходного файла формируется на основе имени входного файла. К исходному имени добавляется суффикс `_compressed`, расширение сохраняется. Далее создается объект FFmpeg, указываются входной файл, выходной файл и рассчитанные параметры кодирования.

4.4 Выбор кодировщика

Модуль `utils.py` отвечает за выбор доступного HEVC-кодировщика. В проекте описаны варианты `hevc_nvenc`, `hevc_qsv`, `hevc_vaapi`, `hevc_amf`, `hevc_vulkan` и `libx265`. Сначала программа получает список доступных кодировщиков FFmpeg, затем проверяет их в порядке приоритета.

Если доступен аппаратный кодировщик, приложение может использовать его. Если подходящий аппаратный вариант не найден, используется программный `libx265`. Для каждого кодировщика заданы собственные параметры качества, так как разные реализации FFmpeg используют разные имена опций.

Такой подход делает приложение более переносимым между компьютерами с разными видеокартами и конфигурациями FFmpeg. При этом пользователю не нужно вручную выбирать конкретный кодировщик.

5 Ход разработки

Разработка проекта выполнялась поэтапно. На первом этапе была сформулирована задача: создать программу для пакетной конвертации видеороликов в более эффективный формат. Был выбран стек Python и FFmpeg, так как Python подходит для быстрой разработки пользовательского приложения, а FFmpeg предоставляет готовые средства обработки видео.

На втором этапе был реализован базовый модуль перекодирования. Он запускал FFmpeg для одного файла, формировал выходное имя и применял параметр качества. После этого была добавлена работа с FFprobe для получения данных о видео: количества кадров, разрешения и частоты кадров.

На третьем этапе была добавлена логика выбора кодировщика. Приложение стало проверять доступные HEVC-кодировщики и выбирать наиболее подходящий вариант. Это позволило поддержать как аппаратное, так и программное кодирование.

На четвертом этапе был разработан графический интерфейс на GTK 4. В него были добавлены выбор файлов, поля настройки качества, разрешения и частоты кадров, строка статуса и индикатор прогресса.

На пятом этапе была реализована пакетная обработка. Пользователь получил возможность выбрать несколько файлов, после чего приложение обрабатывает их последовательно и показывает количество завершенных операций.

6 Тестирование

Тестирование приложения проводилось по основным сценариям использования.

- 1) **Запуск приложения.** Проверялось, что программа открывает главное окно, отображает элементы управления и не завершает работу с ошибкой при наличии необходимых зависимостей.
- 2) **Выбор одного файла.** Проверялось, что после выбора одного видео в интерфейсе отображается имя файла, а кнопка запуска перекодирования начинает обработку.
- 3) **Выбор нескольких файлов.** Проверялось, что после выбора нескольких файлов приложение отображает их количество и последовательно обрабатывает список.
- 4) **Перетаскивание файлов.** Проверялось, что файлы можно добавить переносом в окно приложения.
- 5) **Настройка качества.** Проверялось, что указанное пользователем значение качества передается в параметры кодировщика.
- 6) **Ограничение разрешения.** Проверялось, что при значении 0 сохраняется исходный размер кадра, а при меньшем значении применяется фильтр масштабирования.
- 7) **Ограничение частоты кадров.** Проверялось, что при значении 0 сохраняется исходная частота кадров, а при меньшем значении применяется фильтр fps.
- 8) **Обработка ошибки.** Проверялось, что при ошибке FFmpeg приложение разблокирует кнопки и выводит сообщение о сбое.

По результатам тестирования основные функции приложения соответствуют техническому заданию: файлы выбираются, параметры задаются через интерфейс, перекодирование запускается, прогресс отображается, результат сохраняется в файл с суффиксом `_compressed`.

7 Заключение

В ходе курсовой работы было разработано настольное приложение Floppy для пакетной конвертации видеороликов в эффективный формат хранения. Программа реализована на языке Python с использованием GTK 4 для графического интерфейса и FFmpeg для обработки видео.

В приложении реализованы выбор одного или нескольких файлов, добавление файлов перетаскиванием, настройка качества, ограничение разрешения и частоты кадров, последовательное перекодирование файлов, отображение прогресса и сообщений о результате. Для повышения переносимости реализован автоматический выбор доступного HEVC-кодировщика.

Разработанное приложение решает поставленную задачу: позволяет уменьшать размер видеофайлов без ручного ввода команд FFmpeg. В качестве дальнейшего развития можно добавить сохранение пользовательских настроек, журнал выполненных операций, предварительную оценку размера результата и более подробные сообщения об ошибках.

8 Список использованных источников

- 1 ITU-T. Recommendation H.265: High efficiency video coding. – URL: <https://www.itu.int/rec/T-REC-H.265> (дата обращения 08.05.2026).
- 2 FFmpeg Documentation. – URL: <https://ffmpeg.org/documentation.html> (дата обращения 08.05.2026).
- 3 PyGObject Documentation. – URL: <https://pygobject.gnome.org/> (дата обращения 09.05.2026).
- 4 python-ffmpeg Documentation. – URL: <https://python-ffmpeg.readthedocs.io/> (дата обращения 08.05.2026).

А Руководство пользователя

А.1 Назначение программы

Florpy предназначено для пакетного перекодирования видеороликов в HEVC. Программа помогает уменьшить размер видеофайлов на диске и не требует от пользователя ручного ввода команд FFmpeg.

А.2 Установка

Для работы приложения требуется Linux, Python, GTK 4, FFmpeg и Python-зависимости из файла `requirement.txt`. Перед запуском необходимо установить системные пакеты GTK 4 и FFmpeg средствами пакетного менеджера операционной системы, затем установить зависимости Python.

Пример установки Python-зависимостей:

```
pip install -r requirement.txt
```

А.3 Запуск

Запуск приложения выполняется из каталога проекта:

```
python app.py
```

После запуска открывается окно Florpy с элементами выбора файлов и настройки параметров.

А.4 Основной сценарий использования

- 1) нажать кнопку `Choose files` или перетащить видеофайлы в окно приложения;
- 2) выбрать значение качества;
- 3) при необходимости указать максимальную высоту видео;
- 4) при необходимости указать максимальную частоту кадров;
- 5) нажать кнопку `Reencode`;

- 6) дождаться завершения обработки;
- 7) найти рядом с исходным файлом новый файл с суффиксом `_compressed`.

A.5 Типовые ошибки

Если пользователь нажал `Reencode`, но не выбрал файлы, программа выводит сообщение `Choose files first`. Нужно выбрать хотя бы один видеофайл и повторить запуск.

Если в системе не установлен `FFmpeg` или отсутствует подходящий `HEVC`-кодировщик, перекодирование завершится ошибкой. В этом случае нужно установить `FFmpeg` или включить кодировщик `HEVC` в используемой сборке `FFmpeg`.

Если выбранный файл поврежден или не является видеофайлом, `FFmpeg` может завершить обработку с ошибкой. Нужно проверить исходный файл или выбрать другой файл.

В Руководство разработчика

В.1 Структура проекта

Проект Floppy расположен в каталоге `floppy`. Основные файлы:

- `app.py` – точка входа приложения и описание графического интерфейса;
- `engine.py` – логика перекодирования одного файла;
- `utils.py` – работа с FFmpeg, FFprobe, кодировщиками и параметрами вывода;
- `requirement.txt` – список Python-зависимостей.

В.2 Поток выполнения

При запуске `app.py` создается объект `FloppyApp`, затем главное окно `MainWindow`. Пользователь выбирает файлы, после чего метод обработки кнопки считывает значения интерфейса и запускает отдельный поток. В этом потоке для каждого файла вызывается асинхронная функция `engine.reencode`.

Функция `reencode` получает технические данные исходного видео через функции из `utils.py`, формирует параметры FFmpeg и запускает обработку. Прогресс передается обратно в интерфейс через `callback`. Обновление GTK-виджетов выполняется через `GLib.idle_add`, чтобы изменения происходили в основном потоке интерфейса.

В.3 Выбор кодировщика

Доступные кодировщики описаны в словаре `ENCODERS`. Порядок выбора задан списком `ENCODER_PRIORITIES`. Для добавления нового кодировщика нужно:

- добавить описание в `ENCODERS`;
- указать имя кодека FFmpeg;
- задать параметры качества;
- при необходимости указать аппаратное ускорение и дополнительные параметры;

- добавить ключ кодировщика в `ENCODER_PRIORITIES`.

В.4 Сборка и запуск при разработке

Перед разработкой нужно установить зависимости:

```
pip install -r requirement.txt
```

Также в системе должны быть доступны исполняемые файлы `ffmpeg` и `ffprobe`. Проверить их наличие можно командами:

```
ffmpeg -version  
ffprobe -version
```

Запуск приложения для проверки:

```
python app.py
```

В.5 Возможные направления развития

В дальнейшем разработчик может добавить сохранение пользовательских настроек, выбор каталога для результатов, отображение подробного журнала FFmpeg, отмену текущей операции и отдельный режим оценки ожидаемого размера файла.